

Laboratorio 5: Clasificación de tweets usando minería de texto

CC3084 – Data Science, Semestre II 2026

Anthony Lou Schwank – 23410

Isabella Obando – 23074

Francisco Martinez – 23617

Repositorio del proyecto:

https://github.com/Cisco890/lab5_datascience

1. Introducción

El objetivo del laboratorio fue construir un clasificador capaz de determinar si un tweet se refiere a un desastre real o no, usando el dataset *Natural Language Processing with Disaster Tweets* de Kaggle [1]. El trabajo combinó minería de texto clásica (limpieza, n-gramas, TF-IDF) con clasificación supervisada y análisis de sentimiento, y se organizó en dos notebooks: uno de avance (`lab5_avances.ipynb`) con el análisis exploratorio, el preprocesamiento y el modelo preliminar, y uno de cierre (`lab5_clasificacion_sentimiento.ipynb`) con la función de clasificación, el análisis de sentimiento y la variable de negatividad. Ambos reutilizan el mismo pipeline de limpieza y entrenamiento, centralizado en `src/lab5_pipeline.py` para que los resultados sean reproducibles entre notebooks.

2. Descripción del conjunto de datos

El conjunto de entrenamiento tiene **7,613 tweets** y 5 columnas: `id`, `keyword` (61 valores faltantes, 0.80 %), `location` (2,533 faltantes, 33.27 %), `text` (completo) y `target` (1 = desastre real, 0 = no). La Figura 1 resume los faltantes por columna.

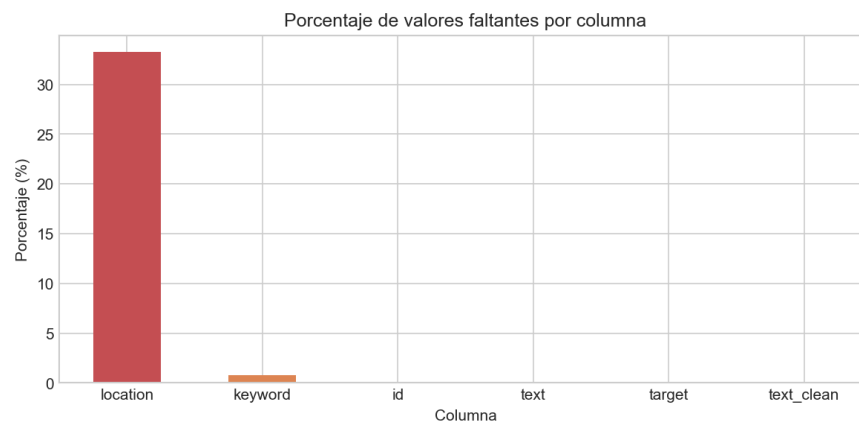


Figura 1: Porcentaje de valores faltantes por columna.

La variable objetivo muestra un desbalance moderado: **4,342 tweets (57.03 %)** de clase 0 y **3,271 (42.97 %)** de clase 1 (Figura 2). No se encontraron filas completamente duplicadas, aunque sí **110 textos repetidos**, que se conservaron para no alterar artificialmente la distribución del dataset.

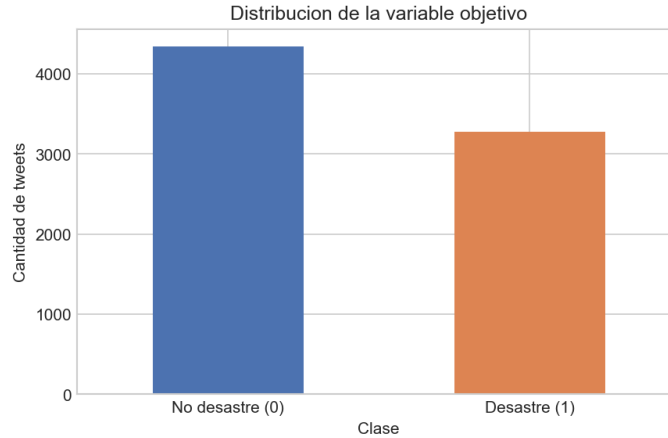


Figura 2: Distribución de la variable objetivo.

3. Preprocesamiento y limpieza de texto

La limpieza (función `limpiar_texto`, implementada con `re` y `scikit-learn`) aplicó, en orden: conversión a minúsculas; eliminación de URLs y menciones `@usuario`; eliminación de emojis Unicode y emoticones de texto (p. ej. `:)`); eliminación del símbolo `#` conservando la palabra del hashtag; eliminación de signos de puntuación y apóstrofes; y eliminación de *stopwords* en inglés (`ENGLISH_STOP_WORDS` de `scikit-learn`). Los números se conservaron deliberadamente: aunque la mayoría son genéricos, casos como **911** aparecen asociados a emergencias reales.

Antes de decidir qué hacer con los emoticones se cuantificó su presencia: no se encontró **ningún** tweet con emoji Unicode real, y solo **82 tweets** (1.1 % del dataset) con emoticones de texto, sin diferencia relevante entre clases. Por eso se eliminaron de `text_clean` (el texto usado por el modelo de bolsa de palabras), pero se conservó el texto original sin modificar para el análisis de sentimiento de la Sección 7, donde sí pueden aportar señal. Tras la limpieza completa solo **4 textos** quedaron vacíos, lo que indica que el preprocesamiento no fue demasiado agresivo.

4. Análisis exploratorio

Los tweets de desastre real tienden a ser ligeramente más largos (108.1 caracteres en promedio vs. 95.7 en la otra clase) y a incluir más URLs (0.77 vs. 0.51 en promedio), mientras que las menciones son más frecuentes en tweets no-desastre (Figura 3). Estas diferencias existen pero no son lo bastante fuertes como para separar las clases por sí solas, por lo que la señal principal debe venir del contenido léxico del texto.

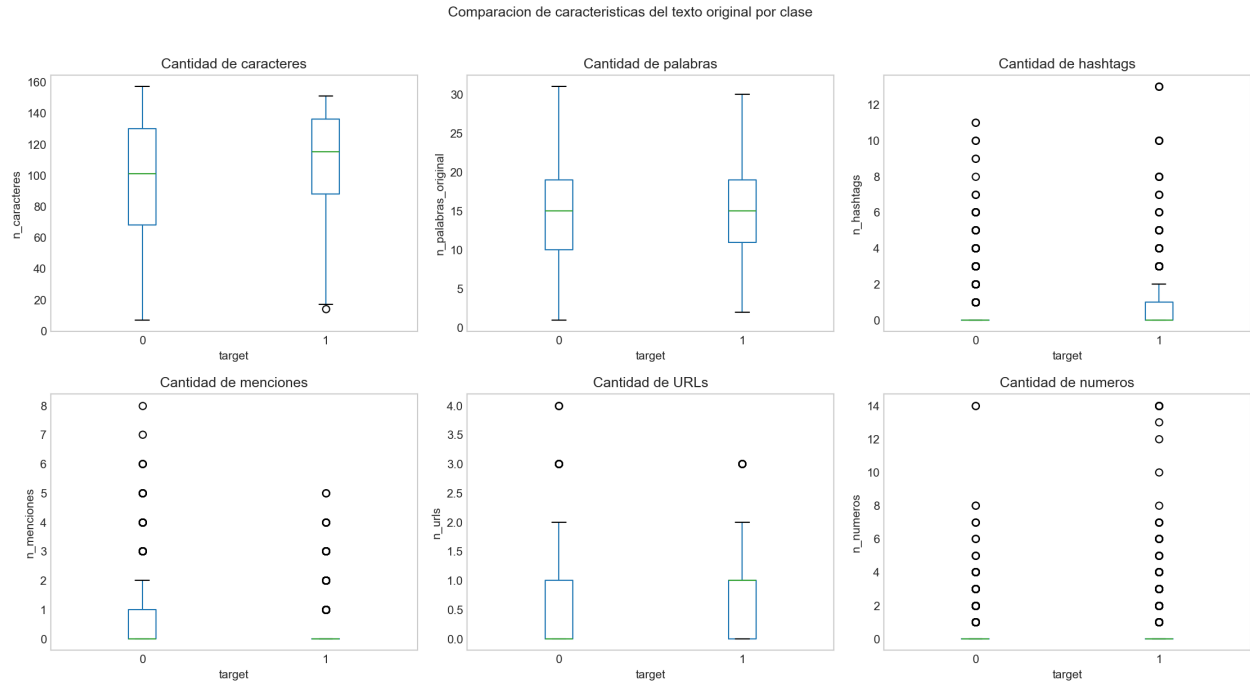


Figura 3: Características del texto original por clase.

4.1. Frecuencia de unigramas y bigramas

La Figura 4 muestra los 20 unigramas más frecuentes por clase. Palabras como *like*, *just* o *new* dominan en tweets no-desastre, mientras que *news*, *disaster*, *california* y *suicide* dominan en tweets de desastre real. Sin embargo, varias palabras frecuentes (*amp*, *people*, *time*) aparecen en ambas categorías, lo que muestra que la frecuencia aislada no basta para separar clases.

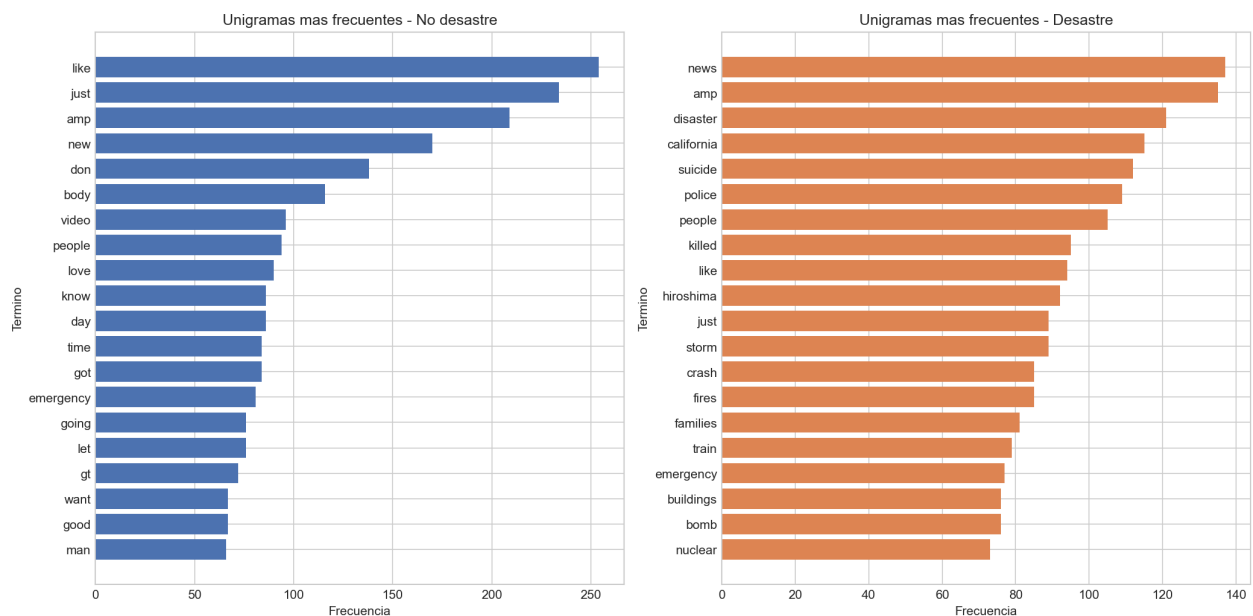


Figura 4: Unigramas más frecuentes por clase.

Los bigramas (Figura 5) sí aportan contexto adicional: expresiones como **suicide bomber**, **california wildfire** u **oil spill** son casi exclusivas de tweets de desastre real, mientras que combinaciones como **cross body** o **liked video** son típicas de la otra clase (p. ej. tweets sobre bolsos/moda o notificaciones automáticas de redes sociales, no sobre desastres). Se exploró también un nivel de tri-gramas, pero con textos tan cortos como los tweets su frecuencia cae rápido y aportan pocas frases nuevas, por lo que se dejaron como análisis exploratorio y no como parte del modelo.

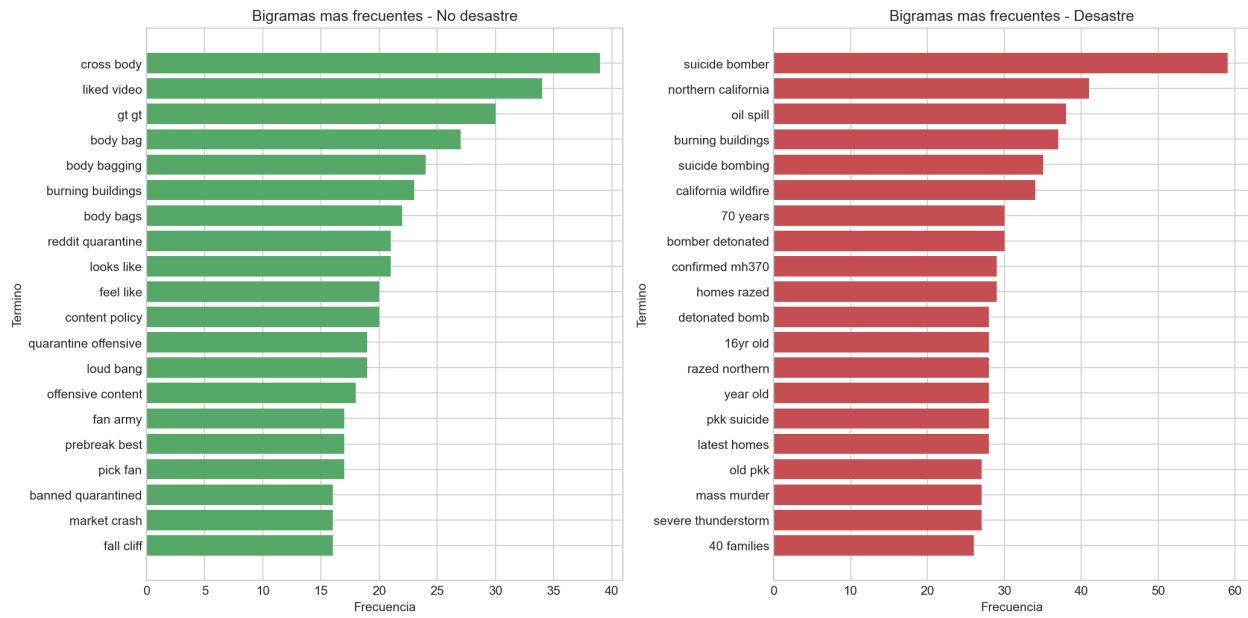


Figura 5: Bigramas más frecuentes por clase.

4.2. Nube de palabras

La Figura 6 visualiza el mismo vocabulario de forma cualitativa. En la nube de “desastre” resaltan términos como **disaster**, **suicide**, **california**, **police** y **killed**; en la de “no desastre” resaltan **love**, **video**, **time** y **new**. Palabras como **amp** y **new** aparecen grandes en ambas, lo que confirma la ambigüedad ya detectada en las tablas de frecuencia.

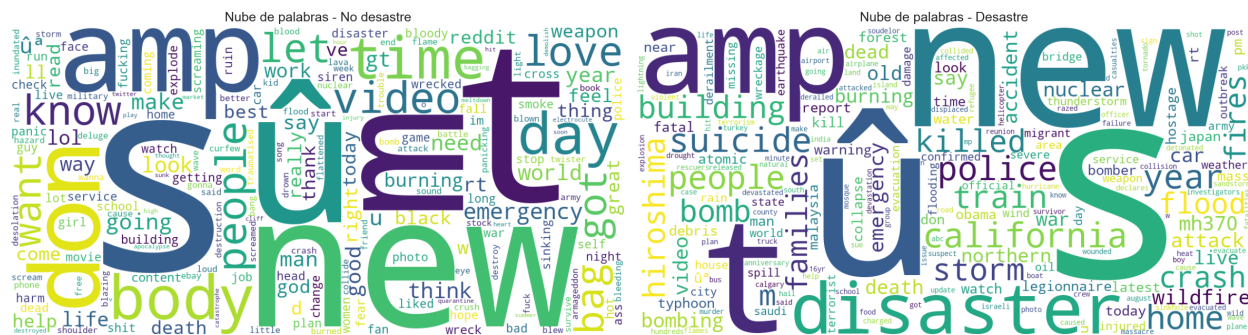


Figura 6: Nube de palabras por clase (generada con la librería wordcloud).

4.3. Probabilidades de n-gramas por clase

Para complementar las frecuencias con una medida de probabilidad, se calculó, para cada uno de los 20 unigramas y 20 bigramas globales más frecuentes, la proporción de tweets de cada clase que lo contienen ($P(\text{término} \mid \text{desastre})$ y $P(\text{término} \mid \text{no desastre})$) y su razón de verosimilitud en escala logarítmica (*log-odds*) –la misma lógica que usa internamente un clasificador Naive Bayes. La Figura 7 muestra el resultado ordenado de menor a mayor *log-odds*.

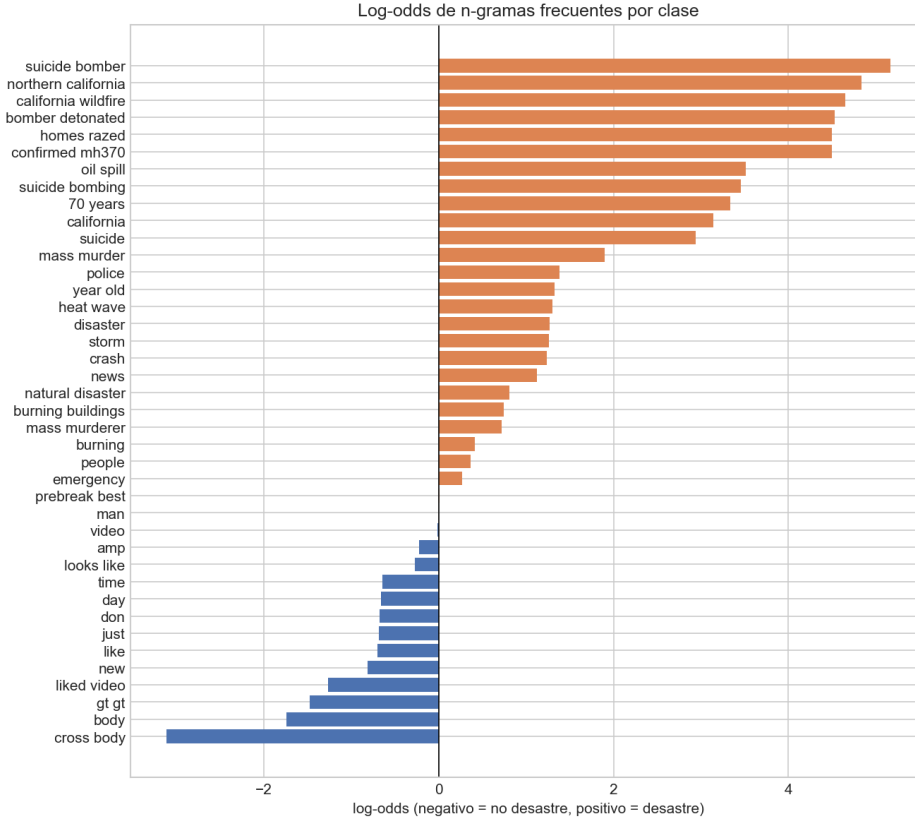


Figura 7: Log-odds de n-gramas frecuentes por clase.

El término más exclusivo de “desastre” fue **suicide bomber** ($P = 1,74\%$ vs. $0,00\%$), y el más exclusivo de “no desastre” fue **cross body** ($P = 0,90\%$ vs. $0,03\%$, proveniente de tweets sobre bolsos y moda que usan la palabra **body** sin relación con desastres). Los términos con *log-odds* cercano a 0 (**amp**, **video**) son frecuentes en ambas clases y por sí solos aportan poca capacidad discriminativa.

4.4. Variable keyword

La columna **keyword** resume un tema explícito del tweet y muestra proporciones de clase muy variables: **derailment** y **wreckage** son 100% desastre real, mientras que **aftershock** es 0% . Es una variable interpretable, pero con ruido suficiente (algunas keywords aparecen en ambos contextos) como para no incorporarla todavía en el modelo de texto puro.

5. Modelos de clasificación

5.1. Representación y algoritmo principal

Como representación se usó **TF-IDF** (`TfidfVectorizer` de `scikit-learn`, `min_df=2`, `max_df=0.95`) sobre `text_clean`, ajustada únicamente con el 80 % de entrenamiento (`train_test_split`, `stratify=y`, `random_state=42`) para evitar fuga de información. El algoritmo principal fue **Regresión Logística** (`max_iter=1000`), elegida por su simplicidad, velocidad e interpretabilidad para datos de texto dispersos.

Cuadro 1: Comparación de representaciones (Regresión Logística).

Representación	Accuracy	Precision	Recall	F1
Unigramas	0.8227	0.8516	0.7110	0.7750
Unigramas + bigramas	0.8234	0.8519	0.7125	0.7760

Agregar bigramas mejoró marginalmente todas las métricas, por lo que se conservó la representación unigramas + bigramas para el resto del análisis.

5.2. Comparación de algoritmos

Para cumplir con probar varios algoritmos, se entrenó además un **Naive Bayes Multinomial** sobre la misma representación. La Tabla 2 y la Figura 8 resumen la comparación.

Cuadro 2: Comparación de algoritmos de clasificación.

Algoritmo	Accuracy	Precision	Recall	F1
Regresión Logística	0.8234	0.8519	0.7125	0.7760
Naive Bayes Multinomial	0.8155	0.8621	0.6789	0.7596

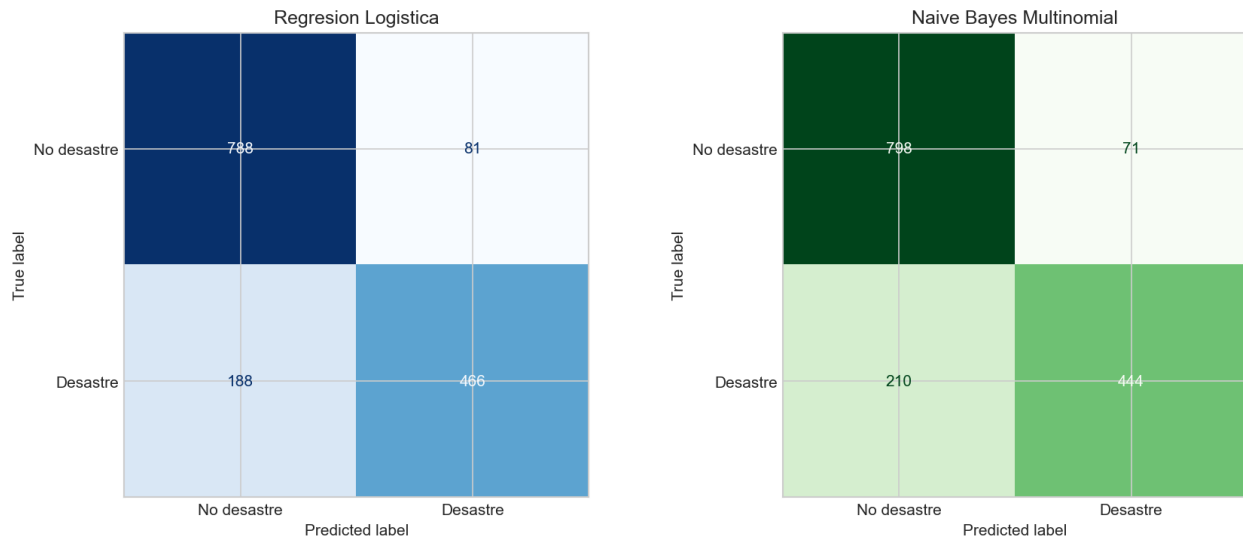


Figura 8: Matrices de confusión: Regresión Logística y Naive Bayes Multinomial.

Naive Bayes tiene mejor *precision* pero peor *recall*: su suposición de independencia entre palabras es una simplificación fuerte cuando el contexto importa (distinguir bigramas con significado literal de uno figurado). La Regresión Logística, al permitir regularización y no asumir esa independencia, obtuvo mejor F1 global y se seleccionó como **modelo final** para el resto del análisis (función de clasificación, interpretación de coeficientes y variable de negatividad).

Los coeficientes de la Regresión Logística confirman los mismos patrones que el análisis de *log-odds*: términos como *hiroshima*, *california*, *fires* y *killed* tienen los coeficientes más altos hacia “desastre”, mientras que *love*, *new* y *let* son los más fuertes hacia “no desastre”. Al revisar una muestra de errores, la mayoría corresponde a tweets ambiguos, cortos o con uso figurado del vocabulario, difíciles de resolver con una representación de bolsa de palabras.

6. Función de clasificación

Se implementó `clasificar_tweet(texto)`, que recibe el **texto crudo de un tweet, sin pre-procesar**, aplica internamente la misma limpieza y el mismo vectorizador TF-IDF del modelo entrenado, y devuelve la etiqueta predicha junto con la probabilidad asociada. La Tabla 3 muestra ejemplos con mayúsculas, menciones, URLs y hashtags sin procesar.

Cuadro 3: Ejemplos de clasificación con texto crudo.

Tweet (texto crudo)	Etiqueta	Prob.
BREAKING: Massive 7.2 earthquake hits California...	Desastre	90.2 %
This traffic jam... is an absolute disaster... lol	Desastre	52.9 %
Forest fire spreading fast near the city...	Desastre	81.9 %
I'm on fire today, nailed every presentation...	No desastre	67.9 %
Just watched a documentary about the 2004 tsunami...	No desastre	70.7 %
@user CAN YOU BELIEVE this flood destroyed...	No desastre	56.2 %

El modelo distingue correctamente los usos figurados más evidentes (“I’m on fire today” sobre una buena jornada de trabajo), aunque casos ambiguos como el uso coloquial de “disaster” para describir tráfico quedan cerca del umbral de decisión (52.9 %), reflejando la dificultad ya observada en el análisis de errores.

7. Análisis de sentimiento

Para determinar si un tweet es positivo, negativo o neutro se usó **VADER** (*Valence Aware Dictionary and sEntiment Reasoner*) [2], disponible en `nltk.sentiment` –uno de los módulos sugeridos en el enunciado. VADER es un analizador basado en léxico diseñado específicamente para texto de redes sociales: cada palabra tiene un puntaje de valencia, y el puntaje agregado (**compound**, entre -1 y 1) pondera además mayúsculas, signos de exclamación, negaciones e intensificadores. Se clasificó cada tweet como positivo si **compound** $\geq 0,05$, negativo si $\leq -0,05$, y neutro en otro caso (umbral recomendado por la documentación oficial de VADER).

A diferencia de `text_clean`, para este análisis se usó una limpieza liviana que solo elimina URLs y menciones, **conservando** mayúsculas, puntuación y emoticones, ya que son señales que el léxico de VADER usa explícitamente.

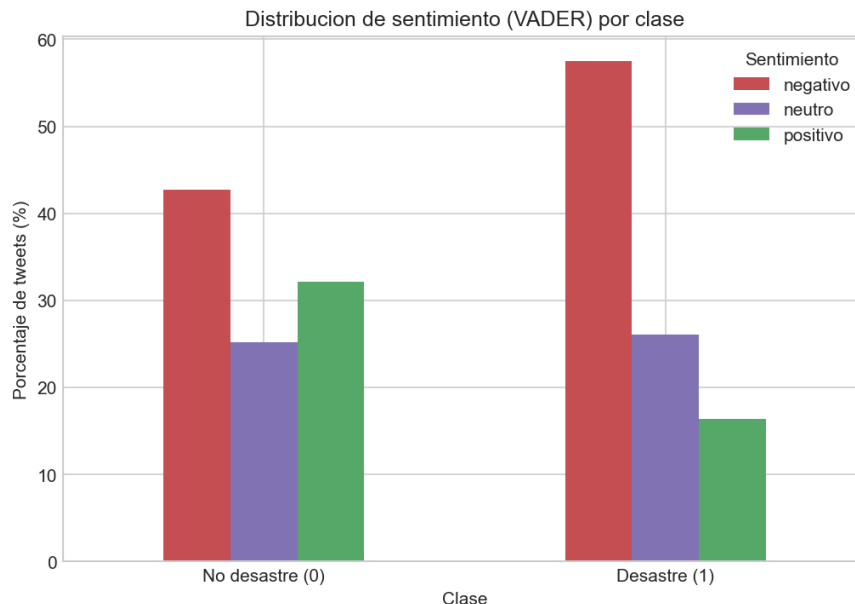


Figura 9: Distribución de sentimiento (VADER) por clase.

Como muestra la Figura 9, los tweets de desastre real son **57.5% negativos**, frente a **42.7%** en la otra clase; el patrón inverso ocurre con los positivos (16.4% vs. 32.1%).

8. Resultados

8.1. Tweets más negativos y más positivos

De los **10 tweets más negativos** según VADER, **8 pertenecen a la categoría “Desastre”**: temáticamente son atentados, bombas suicidas y ataques con víctimas mortales, vocabulario que VADER capta como muy negativo tanto por las palabras (*killed, suicide bomber, dead*) como por señales de énfasis (mayúsculas, repetición, signos de exclamación). De los **10 tweets más positivos**, **8 pertenecen a “No desastre”**: predominan tweets sobre entretenimiento, agradecimientos o buenos deseos (*love, happy, great*).

8.2. ¿Son los tweets de desastre real más negativos?

Sí: el promedio de `vader_compound` es $-0,2668$ para tweets de desastre real frente a $-0,0565$ para el resto, y una prueba t de Welch confirma que la diferencia es estadísticamente significativa ($p < 0,05$). Esto es consistente con lo observado en la Sección 7 y con el hecho de que los tweets sobre desastres reales describen muertes, destrucción y emergencias, mientras que el uso figurado del vocabulario de desastre aparece más en contextos cotidianos o positivos.

8.3. Variable de negatividad y reentrenamiento

Se definió `negatividad` = $-\text{vader_compound}$ (rango $[-1, 1]$, valores altos = tweet más negativo) y se agregó como columna numérica adicional a la matriz TF-IDF (`scipy.sparse.hstack`), estandarizada con `StandardScaler` ajustado solo con el conjunto de entrenamiento. La Figura 10 muestra que, en efecto, la distribución de negatividad es más alta en tweets de desastre real.

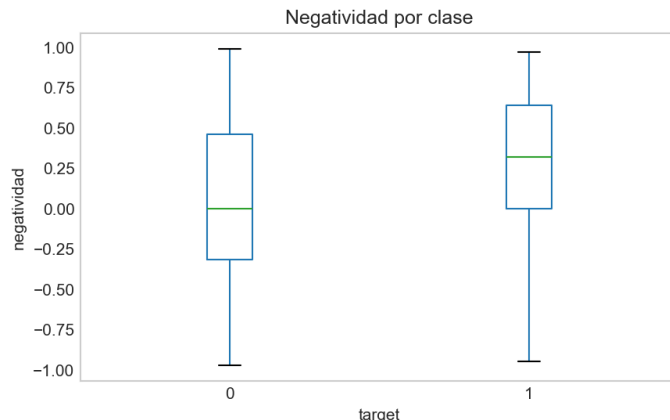


Figura 10: Negatividad por clase.

Cuadro 4: Efecto de agregar la variable de negatividad al modelo.

Modelo	Accuracy	Precision	Recall	F1
Base (solo TF-IDF)	0.8234	0.8519	0.7125	0.7760
+ negatividad	0.8188	0.8399	0.7141	0.7719

El resultado honesto es que **incluir la negatividad no mejoró el modelo**: el F1 bajó de 0.7760 a 0.7719 (accuracy y precision también bajan levemente; solo el recall sube 0.0016). Esto es razonable: la negatividad resume todo el tweet en un solo número, mientras que la representación TF-IDF con miles de términos ya captura señales mucho más específicas (palabras casi inequívocas de una clase, como **hiroshima** o **wildfire**, que ninguna medida de sentimiento general puede replicar). El sentimiento negativo de un tweet ya está correlacionado con varias de esas palabras que el TF-IDF también ve, así que la variable aporta poca información nueva y, en este caso, ni compensa el ruido adicional que introduce.

9. Limitaciones

El dataset tiene 110 textos repetidos y algunos caracteres corruptos (secuencias como $\hat{u}_-$) provenientes de una codificación inconsistente en el origen de los datos, que no se corrigieron por no afectar de forma relevante el vocabulario más frecuente. El umbral de sentimiento de VADER (± 0.05) es el recomendado por la librería, pero no fue calibrado específicamente para este dominio (tweets de desastres). La variable **keyword**, con potencial predictivo, no se incorporó todavía al modelo final. Finalmente, la comparación de algoritmos se limitó a dos familias (Regresión Logística y Naive Bayes); otros algoritmos (p. ej. SVM lineal o *gradient boosting*) podrían mejorar aún más el resultado.

10. Conclusiones

El texto limpio (**text_clean**) con representación TF-IDF (unigramas + bigramas) y Regresión Logística alcanzó **Accuracy = 0.8234**, **Precision = 0.8519**, **Recall = 0.7125** y **F1 = 0.7760** en el conjunto de prueba, superando a Naive Bayes Multinomial (F1 = 0.7596). El análisis de n-gramas y de *log-odds* identificó términos casi exclusivos de cada clase (**suicide bomber** para desastre, **cross**

`body` para no desastre), pero también mostró que buena parte del vocabulario frecuente es ambiguo y depende del contexto, lo que justifica el uso de bigramas. El análisis de sentimiento con VADER confirmó, de forma estadísticamente significativa, que los tweets de desastre real son más negativos que el resto (compound promedio $-0,27$ vs. $-0,06$), aunque convertir ese hallazgo en una variable de negatividad no mejoró el modelo de clasificación final: la señal ya estaba, en su mayoría, capturada por la representación TF-IDF. La función `clasificar_tweet` deja el pipeline completo (limpieza + vectorización + predicción) disponible para clasificar tweets nuevos sin preprocesar.

Referencias

- [1] Kaggle. *Natural Language Processing with Disaster Tweets*. <https://www.kaggle.com/competitions/nlp-getting-started>.
- [2] Hutto, C.J. & Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *Eighth International Conference on Weblogs and Social Media (ICWSM-14)*.
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [4] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [5] Müller, A. *word_cloud: A little word cloud generator in Python*. https://github.com/amueller/word_cloud.